算法设计与分析讲义

Lecture notes on Algorithm Design and Analysis (LADA)

黄宇
南京大学计算机科学与技术学院
https://github.com/alg-nju/lada
2026 年 10 月 15 日

使用说明

- 讲义的一章大致对应于一次授课。每一章的开头会有一个内容概要，列出此次授课所有的知识要点。
- 讲义的内容在持续写作中。目前优先写进讲义的内容，是当前授课所用的课本和胶片中，讲得不足需要补充和调整的部分。也就说，讲义尚未写出来的内容，在课本和胶片中可以找到相应的详细描述。

前言

千教万教，教人求真。千学万学，学做真人。

—— 陶行知

前言。

社会、技术都在剧烈变革，变革中的师、生如何自处、相处，是一个严肃的问题。录陶行知先生名言一句放在开头，与诸位共勉。

目录

第一部分　准备知识 **6**

第一章　计算模型 (L1) **8**

1.1　抽象的计算机：计算模型的引入 8

1.1.1　算法的直观概念 8

1.1.2　Pointer模型：RAM模型的对象化改造 9

第二章　函数的渐近增长率 (L2) **11**

第三章　分治递归求解 (L3) **12**

3.1　Master定理 12

第四章　算法视角下的数学知识 (T1) **15**

第二部分　从蛮力到分治 **17**

第五章　快速排序 (L4) **19**

5.1　快速排序 20

5.1.1　原地地划分的实现 20

第六章　堆排序 (L5) **22**

第七章　归并排序 (L6) **23**

7.1　归并排序的设计 23

7.1.1　归并过程的空间复杂度 23

7.2　归并排序的分析 24

7.2.1　最坏情况下的归并代价 24

7.3　基于比较的排序算法的下界 25

第八章　从排序到分治 (T2) **26**

8.1　归并分治 26

8.1.1　最近点对问题 26

第九章 线性时间选择 (L7, L8) **28**
9.1 分组大小背后的微妙权衡 29

第十章 对数时间查找 (L9) **31**

第十一章 分治的平衡性 (T3) **32**

第三部分 从图遍历到图优化 **33**

第十二章 图遍历 (L10) **35**
12.1 图算法总览 35
12.2 WFS遍历框架 36
12.2.1 WFS的设计 36
12.2.2 从WFS逐步精化 36

第十三章 给定源点的最短路径 **38**
13.1 SSSP问题 38
13.1.1 关于负权边的subtleties 38
13.2 Bellman-Ford算法 39
13.2.1 Ford的generic框架 39
13.2.2 Ford框架视角下的Dijkstra算法 39
13.3 从Ford到Bellman-Ford 40
13.3.1 Pre版本：细化边的遍历 40
13.3.2 Pre2版本：明确RELAX的轮次 40
13.4 用DP细化Bellman-Ford算法的设计 41
13.4.1 从$dist(v)$到$dist_{\leq i}(v)$ 41
13.4.2 动态规划版的Bellman-Ford算法 42
13.4.3 两个细节的改进 42
13.4.4 原地的Bellman-Ford算法 42

第十四章 动态规划 (L19, L20) **44**
14.1 44
14.1.1 44
14.1.2 44

附录 A 阐述的结构 **46**
A.1 引言——从一个普遍的困境出发 46
A.2 理解的前结构——听者从来不是一张白纸 47
A.2.1 核心命题 47
A.2.2 对阐述的启示 47
A.3 认知负荷理论——工作记忆是瓶颈 48

A.3.1 核心命题 48
A.3.2 三类认知负荷 48
A.3.3 对阐述的启示 48
A.4 结构主义教学理论——把阐述做成工程 49
A.4.1 核心命题 49
A.4.2 两个关键机制 49
A.4.3 四条教学原则 → 阐述四原则 49
A.5 总结——三个理论构成一个闭环 50

第一部分

准备知识

准备知识部分。

第一章　计算模型 (L1)

授课提纲

计算模型

- 计算的基本概念
- 抽象算法的概念
- RAM模型的引入
- Pointer模型的拓展[1]

算法设计

- 正确性的标准：问题的规约（specification）
- 证明正确性的手段：数学归纳法
- 示例：Euclid算法的正确性证明

算法分析

- 抽象的算法分析：一个计数（counting）问题
- 时间复杂度分析：最坏情况 vs. 平均情况
- 空间复杂度分析
- 一个简单的示例：数组扫描算法

1.1　抽象的计算机：计算模型的引入

1.1.1　算法的直观概念

我们首先不严格地讨论算法的直观概念。

算法是“the step-by-step procedures that computers use to solve problems” [2] 。

[1]在开始讲图算法时，正式引入。

[2]这一说法源自于Quanta Magazine的文章《Computer scientists establish the best way to traverse a graph》，作者Ben Brubaker，链接： https://www.quantamagazine.org/computer-scientists-establish-the-best-way-to-traverse-a-graph-20241025/。

我们可以借用“类比”（analog）来解释算法相关的概念 [3] 。假设我们了解一台计算机及其上运行的程序，那么我们的“算法”，就是“计算机程序在数学世界中的类比”（mathematical analog of a computer program）。我们描述算法所用的“伪码”，就是“计算机编程语言在数学世界中类比”（mathematical analog of a programming language）。这里所说的伪码是指面向算法描述的，结构化的自然语言，可以是中文、英文等。我们的“计算模型”，就是“现实世界中的计算机在数学世界中的类比”（mathematical analog of a computer）。上述类比，如表1.1所示。

表 1.1: 概念的对比

数学世界的概念	计算机世界的概念
算法	程序
伪码语言	编程语言
计算模型	计算机

1.1.2 Pointer模型：RAM模型的对象化改造

在算法设计与分析的过程中，我们经常需要用到动态分配的数据对象，例如各种链表、以邻接链表表示的图等。基础的RAM模型并不直接支持“寻址”的概念，表达动态数据结构的时候，会非常地不方便。为了便于使用各种常用的动态数据结构，我们引入Pointer模型的概念 [4] 。

在Pointer模型中，我们可以动态分配、修改、释放数据对象（object）。每个数据对象，包含若干个（但是不超过一个常数的上界）字段（field），每个字段可以是一份存储单元中的数据，也可以是一个指针（pointer），指针指向另一个数据对象——这也是Pointer模型名字的由来。我们还特别约定指针可以取特殊值NULL，表示其不指向任何数据对象。

我们可以在基本的RAM模型中，实现上述指针模型。简单而言，我们可以用存储单元的序号（index）来表示对象的位置，进而实现指针的概念。在实际使用中，我们不详细讨论实现细节，只是假设我们的RAM模型具有动态分配数据对象，寻址一个数据对象并进行各种操作的能力。

章节注释

算法（algorithm）一词的由来

花拉子米（780年代-850年代 [5]），波斯数学家。他的《代数学》是在约830年写成的一部数学著作，这是第一本解决一次方程及一元二次方程的系统著作，他因而被称为代数的创造者。代数的英文单词“algebra”就出自于他的著作中对于一元二次方程的一种解法。花拉子米在825年写成的《印度数字算术》（On the Calculation with Hindu Numerals）一书对于印度-阿拉伯数

[3]这里参考了Erik Demaine教授在课堂上的讲授。课程信息： MIT open courseware, Introduction to algorithms (6.006, fall 2011)。链接：https://courses.csail.mit.edu/6.006/fall11/notes.shtml。最后访问日期：2026.8.5。

[4]同样参考了Erik Demaine教授在MIT 6.006算法课程中的相关讲解，详见§1.1.1的脚注。

[5]大约是中国历史上安史之乱之后，唐朝中晚期时代。

字系统在中东及欧洲的传播尤其重要。《印度数字算术》被翻译成拉丁语“Algoritmi de numero Indorum”，而“算法”（algorithm）一词则来自于花拉子米名字的拉丁文音译。

是“模型源自于实践”，还是“模型指导了实践”

在我们的授课过程中，对于算法知识的学习和编程实践之间的关系，我们是有预设的要求的。我们的授课，假设大家有基本的编程知识，并且有过一定量的基础编程实践。在此基础之上，希望我们的算法设计和分析课程能够帮助大家更好地认识到程序的正确运作、高效运作背后的原理。基于这一“设定”，这次课所讨论的作为抽象算法设计与分析之基础的计算模型，更像是从具体的实践中归纳出来的。

深入思考上面的设定，大家有可能会有这样的困惑。讨论算法的源头，一般都会提到欧几里得[6]算法，它大概形成于中国的战国时期。中国的三国时期的数学家刘徽所提出的割圆法，也是一种优美的算法。为什么在一个电都没有的年代，却可以有算法。另一个更著名的算法问题是希尔伯特第十问题 [7] ，它要求寻找判定丢番图方程是否有解的算法。但究竟什么是算法呢？在希尔伯特提出问题之时却并不存在一个明确定义。

最终图灵机的出现，对什么是计算，什么是算法，给出了精确的定义。图灵机首先是一个纯粹的数学抽象，其诞生动机来自数理逻辑内部的问题，而非当时物理计算技术的直接反映。如果非要说有什么”得益于”的地方，那也许是当时的打字机、磁带等日常机械给了图灵关于”读写头+移动纸带”这一直观隐喻。倒是物理的计算机的制造，得益于图灵机相关理论概念的发展。图灵机先定义了通用计算的数学边界，后来的冯·诺依曼架构、存储程序计算机，在概念上受到了图灵抽象的间接影响。图灵本人二战后参与建造的 ACE（Automatic Computing Engine）明确借鉴了他在1936 年提出的图灵机相关的理论。

所以回到我们的这次课程。从计算机科学技术的历史来看，我们更应该理解为抽象的模型指导了具体的实践。但是在我们今天学习这门本科生基础课程的时候，我们已经站在了巨人的肩膀上。此时，我们的授课方法是，以充分的实践为诱因和助力，更好得学习和理解算法的知识。

[6] Euclid，公元前325年——公元前265年

[7] 关于希尔伯特第十问题的材料有多种，这里主要参考了维基百科和卢昌海老师的《Hilbert第十问题漫谈》

第二章　函数的渐近增长率 (L2)

授课提纲

基本概念

- 渐近增长率概念的由来
- O的定义：基于$\epsilon - N$语言的定义 vs. 基于求极限的定义
- Ω和Θ的定义：由O的定义派生而来
- o和ω的定义：强调档次上的差距
- 经典算法代价函数的分档：$\log n, n, n\log n, n^2, n^3(n^k), 2^n, n!$

Brute-Force（BF）范式

- BF范式的基本概念：以排序、选择、查找为例
- BF范式的方法学意义：认识问题的难度，寻找改进的机会；以算法分析为“导航”
- 经典例题：数组左右交换问题；最大和连续子串问题

章节注释

微积分严格化的历史

$\epsilon - \delta$语言的背后，是微积分严格化的艰苦历程。

牛顿和莱布尼茨虽然把微积分系统化，但是它还是不够严谨。可是当微积分被成功地用来解决许多问题，却使得十八世纪的数学家偏向其应用，而甚少致力于其严谨性。一些数学家，包括科林·麦克劳林，试图证明使用无穷小是可靠的做法，但直到150多年之后才得以成功。

奥古斯丁·路易·柯西和卡尔·魏尔斯特拉斯的工作，实现了对无穷小的记号的回避。微分和积分的基础终于被打下了。在柯西的著作中，可以看到一系列的基础进路尝试，包括通过无穷小来对连续进行定义，和在微分定义中一个不太精确的函数极限原型。而在魏尔斯特拉斯的著述中，对极限概念作了形式化，回避了无穷小的使用。继魏尔斯特拉斯之后，微积分就常以极限作为基础，而非无穷小了。

第三章　分治递归求解 (L3)

授课提纲

分治策略的基本概念

- 递归，作为一种思维模式；解递归方程，作为一种算法分析手段
- 从简单递归到分治策略
- 分治的典型模式（难分易合、易分难合）与样例

解递归的基本技术

- 忽略取整，笨展开
- Guess & Prove：一种定制版的数学归纳法

分治递归求解

- 笨展开+Recursion Tree
- Master定理，三种情况的由来，等比数列：公比大于1、等于1、小于1
- Case 3的进一步解释

3.1　Master定理

定理 3.1 (Master定理). 令$a \geq 1$和$b > 1$为常数，$f(n)$为一定义于非负整数上的函数。$T(n)$为定义于非负整数上的递归函数：

$$T(n) = aT\left(\frac{n}{b}\right) + f(n) \tag{3.1}$$

递归式中的$\frac{n}{b}$指的是$\lfloor \frac{n}{b} \rfloor$或者$\lceil \frac{n}{b} \rceil$。$T(n)$的渐近增长率为：

1. 如果存在某个常数$\epsilon > 0$，使得$f(n) = O(n^{\log_b a - \epsilon})$，则$T(n) = \Theta(n^{\log_b a})$。
2. 如果$f(n) = \Theta(n^{\log_b a})$，则$T(n) = \Theta(n^{\log_b a} \log n)$。
3. 如果存在常数$\epsilon > 0$，使得$f(n) = \Omega(n^{\log_b a + \epsilon})$，且存在某个常数$c < 1$，使得对所有充分大的$n$，$af(\frac{n}{b}) \leq cf(n)$，则$T(n) = \Theta(f(n))$。

证明. 根据图3.1，我们有：

$$T(n) = \underbrace{\Theta(n^{\log_b a})}_{\text{叶节点层代价}} + \underbrace{g(n)}_{\text{所有非叶结点层代价}} \tag{3.2}$$

$$g(n) = \sum_{j=0}^{\lfloor log_b n \rfloor} a^j f(\frac{n}{b^j}) \tag{3.3}$$

对于$g(n)$的求和形式，注意当j取0时，我们得到$f(n)$。此外，求和的项数必须是整数。 递归树的层数记为$\lfloor \log_b n \rfloor + 1$，则非叶子节点有$\lfloor \log_b n \rfloor$层，即$g(n)$的求和形式有$\lfloor \log_b n \rfloor$项。

第一种情况. 已知$f(n) = O(n^{\log_b a - \epsilon})$，所以$f(\frac{n}{b^j}) = O((\frac{n}{b^j})^{\log_b a - \epsilon})$。据此，我们有$g(n)$满足：

$$\begin{aligned}
g(n) &= \sum_{j=0}^{\lfloor log_b n \rfloor} a^j (\frac{n}{b^j})^{\log_b a - \epsilon} \\
&= n^{\log_b a - \epsilon} \sum_{j=0}^{\lfloor log_b n \rfloor} (\frac{ab^\epsilon}{b^{\log_b a}})^j \\
&= n^{\log_b a - \epsilon} \sum_{j=0}^{\lfloor log_b n \rfloor} (b^\epsilon)^j \\
&= n^{\log_b a - \epsilon} (\frac{b^{\epsilon \log_b n} - 1}{b^\epsilon - 1}) \\
&= n^{\log_b a - \epsilon} (\frac{n^{\epsilon - 1}}{b^\epsilon - 1})
\end{aligned}$$

上面的推导过程中，为了表示的方便，我们省去了$g(n)$表达式中的渐进增长率符号$O(\cdot)$。因为b和ϵ为常数，所以我们有$g(n) = O(n^{\log_b a - \epsilon} n^\epsilon) = O(n^{\log_b a})$。所以根据公式3.2，我们证明了$T(n) = \Theta(n^{\log_b a})$。

第二种情况. 已知$f(n) = \Theta(n^{\log_b a})$，则$f(\frac{n}{b^j}) = \Theta((\frac{n}{b^j})^{\log_b a})$。据此，我们有$g(n)$满足：

$$\begin{aligned}
g(n) &= \sum_{j=0}^{\lfloor log_b n \rfloor} a^j (\frac{n}{b^j})^{\log_b a} \\
&= n^{\log_b a} \sum_{j=0}^{\lfloor log_b n \rfloor} (\frac{a}{b^{\log_b a}})^j \\
&= n^{\log_b a} \sum_{j=0}^{\lfloor log_b n \rfloor} 1 \\
&= n^{\log_b a} \log_b n
\end{aligned}$$

上面的推导过程中，为了表示的方便，我们同样省去了$g(n)$表达式中的渐进增长率符号$\Theta(\cdot)$。根据公式3.2，我们证明了$T(n) = \Theta(n^{\log_b a} \log n)$。

第三种情况. 根据$T(n)$的原始表达式（公式3.1），我们知道$T(n) = \Omega(f(n))$。所以，为了证明$T(n) = \Theta(f(n))$，我们的任务只剩下证明$T(n) = O(f(n))$。根据第三种情况的第一个条件：存在常数$\epsilon > 0$，使得$f(n) = \Omega(n^{\log_b a + \epsilon})$。根据$T(n)$的递归树展开表达式（公式3.2），我

们有$T(n)$的第一项$n^{\log_b a} = O(f(n))$。此时我们的任务只剩下证明$T(n)$的第二项$g(n) = O(f(n))$。而这一结论，完全是第三种情况的第二个条件，给“硬性规定出来”的。

对于第三种情况，已知$af(\frac{n}{b}) \leq cf(n)$，所以 $a^2 f(\frac{n}{b^2}) = a \cdot af(\frac{n/b}{b}) \leq a \cdot cf(\frac{n}{b}) \leq c^2 f(n)$。上述展开可以作用任意$j$次：$a^j f(\frac{n}{b^j}) \leq c^j f(n)$。所以根据公式3.3，我们有$g(n)$满足：

$$
\begin{aligned}
g(n) &= \sum_{j=0}^{\lfloor log_b n \rfloor} a^j f(\frac{n}{b^j}) \\
&\leq \sum_{j=0}^{\lfloor log_b n \rfloor} c^j f(n) \\
&\leq f(n) \sum_{j=0}^{\infty} c^j \qquad (c < 1) \\
&= O(f(n))
\end{aligned}
$$

由此，我们证明了$T(n) = \Theta(f(n))$ [1] 。

□

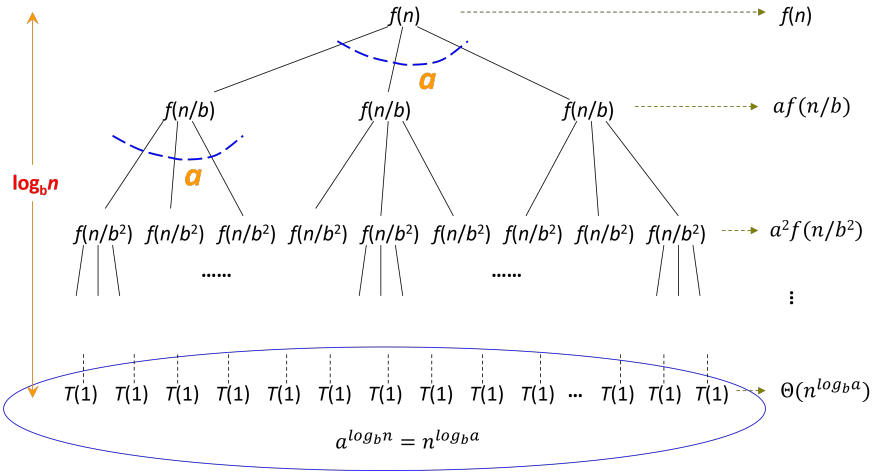


图 3.1: $T(n)$的递归树展开

[1] 证明过程参考了《算法导论》的第三版和第四版的相关章节。图示基于陈道蓄老师的授课胶片修改。

第四章　算法视角下的数学知识 (T1)

If you find the mathematics difficult, skip those early parts. Later sections will be understandable provided you are willing to forgo the deep insights mathematics gives into the weaknesses of our current beliefs. General results are always stated in words, so the content will still be there but in a slightly diluted form.

— Richard W. Hamming [2]

授课提纲

I - 公理化视角

从直觉开始构造

- 数学归纳法的基础：良序原理
- 自然数集合的公理化构造
- 数域的公理化构造：$N \Rightarrow Z \Rightarrow Q \Rightarrow R \Rightarrow C$

数学归纳法证明算法正确性

- 归纳法辨析：“所有马同色”谬误
- 归纳法应用：Euclid，BubbleSort，Prim

认识和理解不变式（invariant）的概念

- 归纳法中的“变”与“不变”
- 更大范围地应用不变式的概念

II - 数学的概念，算法的视角

基本数学概念

- 函数：$f(n)$
- 取整：$\lfloor x \rfloor, \lceil x \rceil$

- 对数：$\log_a b$
- 阶乘：$n!$

基本级数

- 多项式级数：算术级数，平方和，任意k次方和（的渐近增长率）
- 几何级数：一般形式，2和$\frac{1}{2}$的特例，渐进增长率的速判
- 算术几何级数：“错项相减”法
- 调和级数
- 斐波那契数列

面向平均情况时间复杂度分析的期望值求解

- 指标随机变量
- 期望的线性性（linearality）
- 示例：排序算法平均情况时间复杂度的数学描述

III - BF范式：分析驱动设计改进

BF范式（paradigm）

- BF是一个“相对性”的概念
- 基本模式：分析发现不足 $\Rightarrow$ 大O度量不足$\Rightarrow$不足推动改进

分析驱动改进

- “名人”问题；不变式：一次比较，确定淘汰一个
- 频繁项问题：$k=2$的特例；不变式：$|f| > |\neg f|$

第二部分

从蛮力到分治

从蛮力到分治。

第五章　快速排序 (L4)

授课提纲

I - BF排序

最朴素的SelectionSort和BubbleSort

- $W(n)$：$\binom{n}{2}$的必然性
- $A(n) = W(n)$

InsertionSort的改进

- 局部有序性的利用
- $W(n)$：不变
- $A(n)$：定量地降了一半

蛮力排序的不足

- inversion的概念
- inversion的数量：最多、平均
- 蛮力算法：消除inversion速度的不足

II - 快速排序

快速排序的设计原理

- 如何一次比较，消除更多的inversion
- 分治策略的典型应用：难分易合

快速排序的时间复杂度

- $W(n)$：最不平衡的划分
- $A(n)$：列递归，解递归
  - smart guess，guess & prove

– 直接求解：错项相减

- $A(n)$：指标随机变量、变量的linearality [1]

III - 空间复杂度

空间复杂度的分档

- $O(1)$的含义：原地（in-place）排序
- $O(n)$的含义

蛮力排序、堆排序[2]

- 显然的$O(1)$空间

快速排序

- $O(n)$空间的朴素实现
- $O(1)$空间的改进：四个部分、一个不变式

归并排序：多次的反转 [3]

- 只能$O(n)$
- 可以$O(1)$
- 还是$O(n)$

5.1　快速排序

5.1.1　原地地划分的实现

在做Partition的时候，开辟一个$O(n)$的辅助空间，划分很容易实现。一个自然的想法是，能否将划分改为原地（in-place）的。

原地划分的做法是，用两个指针i和j，将元素划分为四个部分，如图5.1所示。划分算法维护的不变式是i和j界定了小于部分和大于部分。

算法当前待处理的元素是$A[j]$。其他情况简单，唯一需要说明的是当$A[j]$纳入一个小于pivot的元素时，不变式被违反。

此时的做法很巧妙：让i指针右移一位，它也违反了不变式。让i、j位置的元素互换，则两个违反都消失，整个数组恢复保持不变式。

[1]将在tutorial（第八章）中专题讲解。

[2]将在相应课程（第六章）中针对性讲解。

[3]将在相应课程（第七章）中针对性讲解。

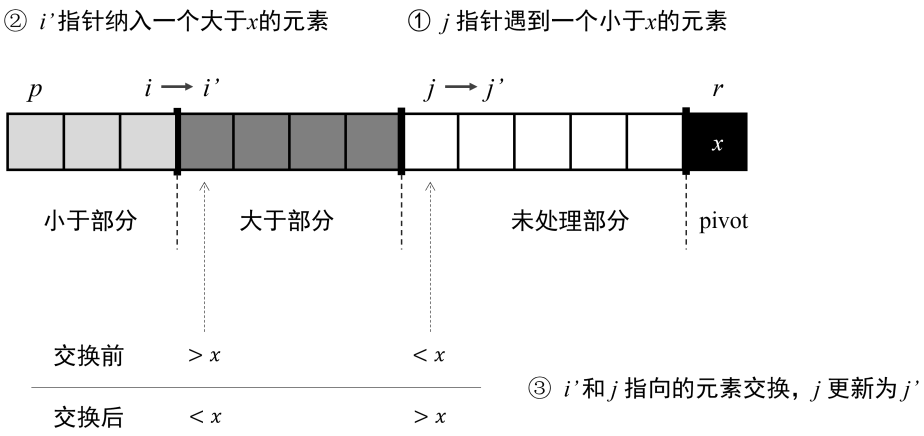

② i'指针纳入一个大于x的元素
① j 指针遇到一个小于x的元素
p
i → i'
j → j'
r
x
小于部分
大于部分
未处理部分
pivot
交换前
> x
< x
交换后
< x
> x
③ i'和 j 指向的元素交换，j 更新为 j'

图 5.1: 原地的元素划分

第六章　堆排序 (L5)

授课提纲

堆的定义

- 二叉树的相关定义：递归的思维方式与定义手段
- 堆的定义：两个维度的解耦

堆算法

- 堆排序
- 堆的修复
- 堆的构建
- 堆结构的数组表示

第七章　归并排序 (L6)

授课提纲

I - 归并排序

MergeSort的设计辨析

- 基本设计与分析
- 合并的最坏情况时间复杂度的问题（§7.2.1）

整体性地考虑空间复杂度问题

- 从蛮力排序和堆排序，到快速排序，到归并排序
- 从$O(n)$到$O(1)$，再到$O(n)$

II - 比较排序的下界

下界问题

- 概念辨析：最大的最小值；最小的最大值
- 下界的定义：最坏情况、平均情况

一个算法族的抽象建模

- 比较排序⇌决策树，对应表（§7.3）

比较排序的下界

- 最坏情况：二叉树的h和L的关系
- 平均情况：越平衡、EPL越小

7.1　归并排序的设计

7.1.1　归并过程的空间复杂度

随机存储（数组） vs. 顺序存储（链表）

数组支持随机访问（random access），即访问任意$A[i]$都是$O(1)$时间。但是数组是一段连续的存储空间，必须预先分配指定的大小。数组中插入元素时，需要涉及多个元素的挪动。

反之，链表只支持顺序访问（sequential access），访问链表中第k个元素的代价为$O(k)$时间。但是链表支持更灵活的调整，插入、删除元素都是$O(1)$时间。链表可以在使用时，即时分配。

$O(n)$的空间代价

对照分析Merge和Partition算法，我们发现由于被Merge的数组是有序的，所以他们无法随意交换元素位置，因而无法像Partition那样，简单地改成原地算法。这与§5.1.1的设计形成鲜明对比。

一种将空间代价降为$O(1)$的“改进”

此时，一个看似合理的改进是：原地Merge，数组被分为左右两半，分别记为A和B。当B头部的元素b_1更小时，我们可以用大约$\frac{n}{2}$次Swap，将b_1向左挪到合适的位置。通过多次的Swap，我们实现了原地的Merge。由于我们是基于比较的排序，上述的Swap操作是完全“免费”的，所以改进后的Merge算法，时间复杂度不变，空间复杂度变为$O(1)$。

这一分析结论过程的问题，在于忽视了我们对critical operation的要求。comparison之所以能够成为critical operation，一个必要的前提是，其他non-comparison的辅助操作的个数，必须跟comparison操作的个数满足线性关系。由此条件，我们才可以（在big-oh的意义下）仅计数 comparison 操作。

上述做法，1次comparison，要辅以$O(n)$次Swap操作，那它已经不再是comparison-based sorting，我们的分析就无效了。如果坚持这样的“改进”，那我们的建模和分析则应该同时统计 Compare 和 Swap 操作的代价。分析结果表明这样的改进就不再有效了。

真的改进，却不是最广泛使用

对于链表，归并是可以在$O(1)$空间完成的。一个比较著名的例子是Linux kernel中对于linked list的归并排序 [1]。

在实际使用中，归并排序算法，主要面向顺序存储上的数据排序。在随机存储上，一般快速排序、堆排序具有更好的性能。虽然上述$O(1)$空间的改进可以实现，但是实际采用的，更多还是需要$O(n)$空间的简单实现 [5]。

7.2　归并排序的分析

7.2.1　最坏情况下的归并代价

归并的最坏情况所对应的输入

假设我们将原始数组分成左右两部分，分别为$A[1..\frac{n}{2}]$和$B[1..\frac{n}{2}]$，现在我们要将他们合并到空数组C中。

对于Merge过程而言，最坏情况输入是A和B的元素在大小关系下是交叉的，即：

$$b_1 < a_1 < b_2 < a_2 < \cdots < b_{n/2} < a_{n/2} \tag{7.1}$$

反过来说，MERGE的最好情况是，有尽量多的元素无需比较，直接完成合并，即：$a_1 < a_2 < \cdots < a_{n/2} < b_1 < b_2 < \cdots < b_{n/2}$。此时，所有$A$的元素通过比较，都挪到$C$中。当$B$中的元素面对空的$A$数组时，无需任何比较，我们直接将整个$B$挪到$C$中。

一种似乎也是最坏的输入

此时，一个潜在的疑问是，公式（7.1）的要求是充分的，但是它否有必要。只要我保证A和B都一直不空（直至最后才空），是否就可以？

假设A和B的元素满足$a_1 < a_2 < \cdots < a_{n/2-1} < b_1 < b_2 < \cdots < b_{n/2} < a_{n/2}$。这一构造的原理是：$A$数组在比较中，差点就空了，只剩一个元素$a_{n/2}$特别大，不是也能保证最坏情况么？因为$B$的元素还是只能通过比较，逐个挪到$C$中。

下面的例子说明，公式（7.1）的要求是必要的，因为面对有序的数组，我们可以玩出一些特别的“花样”。

这样的输入做不到最坏

此时，$a_{n/2}$这一个元素，面对一个有序的数组$B[1..n/2]$，我们可以用折半查找找到$a_{n/2}$应该出现的位置，然后无需额外比较，直接把B和$a_{n/2}$都挪到C中。

此时，基于折半查找，实际的比较次数大约是$\frac{n}{2} + \log \frac{n}{2}$。所以这一情况，由于我们可以采用折半查找，所以它并未将MERGE逼迫到最坏情况。

7.3　基于比较的排序算法的下界

分析比较排序算法的下界，基本的技术手段是，将比较排序算法的运作，和决策树（decision tree）对应起来。这一对应关系，如表7.1所示。

表 7.1: 决策树与比较排序算法的对应关系

Decision Tree	Algorithm
internal node	binary decision (comparison)
leaf	found answer
root-to-leaf path	algorithm execution
path length	running time
height of the tree	worst-case running time

第八章　从排序到分治 (T2)

授课提纲

深入理解排序

- 冒泡排序的改进
- 快速排序的平均情况时间复杂度：基于指标随机变量的分析
- 堆结构背后的深意：前k大元素；$\sum h \leq n-1$

从排序泛化到分治

- inversion计数问题
- 频繁项问题（族）
- 任意精度整数相乘问题；矩阵相乘问题
- 最近点对问题

8.1　归并分治

8.1.1　最近点对问题

我们可以保守地估计，当前检查的点q_i上下所有跟它距离可能更近的点，也就是当前检查的点q_i在一个4×4的网格中，最多需要比较它周边15个点 [3, §5.4]。

大家对于这一更保守的估计反而有点疑惑，就是它为什么考虑了很多显然可以约减的情况。针对这一情况，我们改用下面更细致一点的估计。 q_i只需要考虑它下面的点，连q_i自己，一共8个格子，所以q_i至多需要考虑7个点。

章节注释

“我心目中的英雄”

Karatsuba算法我讲了很多年了，很晚才知道它的名字和背后的“科学轶事”[4]。从维基百科上摘录在下面，供大家参考。我想这则轶事完全可以入选《Heros in My Heart》 [1] 续篇。

[1]《Heros in My Heart》是我学生时代读的一系列BBS文章的合集，对我影响很大。文章的前言转录如下，全文大家可以去网上搜索。“在北大混了四年，一事无成。在未名上 bbs 也呆了快一年了，制造了几千篇的垃圾。要毕业的人想法总是奇怪的，譬如说竟然

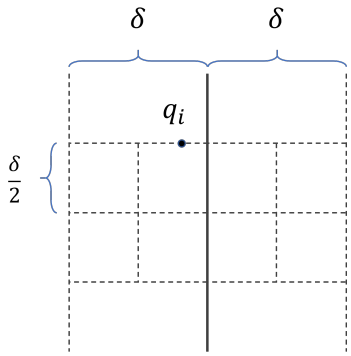


图 8.1: 最近点对问题

*1960*年，科尔莫戈罗夫在莫斯科大学组织了一个关于控制论中数学问题的研讨会，在会上他提出了$\Omega(n^2)$猜想以及计算复杂性的其他一些问题。不到一周，当时还是*23*岁学生的卡拉楚巴便找到了一种算法，仅用$O(n^{\log_2 3})$次初等运算即可完成两个 n 位数相乘，从而推翻了该猜想。

科尔莫戈罗夫对这一发现感到非常兴奋。他在研讨会的下一次会议上通报了这一成果，随后研讨会便宣告结束。科尔莫戈罗夫在世界各地的会议上就卡拉楚巴的成果作了一系列报告（例如见《*1962*年国际数学家大会论文集》第*351–356*页，以及《*1962*年斯德哥尔摩国际数学家大会上的*6*场演讲》），并于*1962*年在苏联科学院院报上发表了这一方法。该文由科尔莫戈罗夫撰写，其中包含关于乘法的两项成果——卡拉楚巴的算法和尤里·奥夫曼的另一项独立成果，文章署名“*A. Karatsuba and Yu. Ofman*”。而卡拉楚巴直到收到出版方寄来的抽印本时，才知道有这篇文章。

真的要正经的写几篇文章了。最初写成这些东西的时候，我发给了几个朋友，一个学数学的师弟说他很感动，一个非数学系的mm说她后悔当初没有选数学系，无论怎样，他们能这样子讲，我很感动，这是发自内心的那种。现在的打算是每天贴2-3个故事，一直到欧毕业那天。很多事情难免有些 too old，这个我也没有办法，激动人心的事情毕竟只有那么多。不多说了，真心的希望大家会喜欢，哪怕只有一点点的喜欢。这些文字偶给了一个名字，叫做我心目中的英雄（Heroes in My Heart）。”

第九章　线性时间选择 (L7, L8)

授课提纲

I - 选择算法

选择问题的特例

- SelMax
- SelMaxMin
- SelSecondMax

期望线性时间选择SelELin

- 脱胎于QuickSortPartition的选择算法
- 时间复杂度：smart guess；严格分析★（两种不同的方法）

最坏线性时间选择SelWLin

- 设计原理：最坏情况下平衡性的保持
- 时间复杂度：guess and prove；基于递归树，笨展开并统计
- 分组大小为5，背后的深意（§9.1）
- 选择问题的泛化：带权重的中位数选择问题

II - 选择下界

对手论证（adversary argument）

- 基本原理
- 基本形式
- 示例——选择唯一不同的球

选择问题的下界

- 选最大
- 选次大
- 选中位数（n为奇数的部分结果）

9.1　分组大小背后的微妙权衡

我们将期望线性时间的选择算法记为SelElin，将最坏线性时间的算法记为SelWlin。

在SelWlin算法中，有一个“神奇”的数字“5”。想要深入理解SelWlin 算法，一个自然的疑问就是，为什么元素的分组大小应该是5？或者说，其他的分组大小——例如分组大小为3、4、7、9等——行不行？

看起来分组的大小有无数种选项，但是我们可以经过简单的分析，将关注点聚焦到少数代表性选项，这可以帮助我们有效理解分组大小“5”背后的原理。首先不难理解，偶数的分组大小可以被排除，因为每组有奇数个元素时，不仅中位数的定义更加简洁，而且更有利于保证最终划分的平衡性。其次，分组大小的选择，以5为基准，有两个方向，一个方向是将分组变得更“细”，此时只有分组大小为3这一个选择。另一个方向是将分组变得更“粗”，这有无穷多种选择，但是我们可以对分组大小7做一个深入剖析，进而理解分组大小选择5的原理。所以我们下面对代表性的分组大小3和7，进行深入分析。

分组大小为3的情况

如果我们将SelWlin算法中分组的大小定为3，而算法的其他设计保持不变，此时算法的运作更接近于SelElin中所有元素一字排开的情况。考虑到SelElin算法的最坏情况时间复杂度$O(n^2)$，我们有理由担心算法的最坏情况时间复杂度是否能保持在$O(n)$的水平。基于上面的考虑，我们试图去证明算法时间复杂度的一个下界，若能证明算法的时间复杂度严格超过了$O(n)$，则可以表明分组大小3不可行。

根据算法的划分方式，我们可以确定比m^*小的元素，至多有：

$$2 \times \left\lceil \frac{1}{2} \left\lceil \frac{n}{3} \right\rceil \right\rceil \leq \frac{n}{3} + 3$$

上述不等式左边的原理很简单，大致估算下来，所有分组中，有一半的小组，每个小组有2个元素，确定比m^*小。由于此时我们只需要估算一个上界，所以我们在除以2或者3的时候，都取上取整，在计算小组数量时，也未扣除m^*所在的小组。与Sel5算法的分析类似，此时最坏情况下，我们需要对至少$n - (\frac{n}{3} + 3) = \frac{2}{3}n - 3$个元素进行递归。所以我们有算法的最坏情况时间复杂度满足：

$$W(n) \geq W\left(\left\lceil \frac{n}{3} \right\rceil\right) + W\left(\frac{2}{3}n - 3\right) + \Omega(n)$$

$W(n)$递归不等式右边的第一项，是对所有小组的中位数，选择m^*的代价；第二项是最坏情况下，对最多的元素进行递归选择的代价；第三项是元素划分的代价。通过“guess and prove”的方式我们不难证明$W(n) = \Omega(n \log n)$，具体证明留作习题。

上述分析表明，分组大小若是设为3，则$W(n)$的渐进增长率严格高于$O(n)$，所以分组大小为3的选项在设计SelWlin算法时，被合理抛弃。

分组大小为7的情况

以分组大小5为基准，更大的取值代表着更精细的分组和更好地平衡性控制（当n充分大的时候），所以我们有理由相信算法的最坏情况代价仍然能保持$O(n)$的水平，而不会出现分组大小为3时，$W(n) = \Omega(n \log n)$的情况。基于这一合理猜测，我们来严格证明$W(n) = O(n)$。

考虑7个元素一小组的划分方式，则确定比m^*小的元素至少有：

$$4 \times \left(\left\lceil \frac{1}{2} \left\lceil \frac{n}{7} \right\rceil \right\rceil - 2 \right) \geq \frac{2}{7}n - 8$$

上述不等式左边的原理是，我们大致有一半约$\frac{n}{7} \times \frac{1}{2}$个小组，每个小组贡献4个元素（每组上方有3个元素，再加上组内的中位数）确定比m^*小。在处理取整的时候，我们为了确定得到一个下界，所以保守地去掉了m^*所在的小组，也去掉了最后一个可能并不完整的小组（由于n可能不是7的倍数，所以最后一小组的元素可能不足7个），所以共去掉2个小组。

所以在最坏情况下，算法要对不超过$\frac{5}{7}n + 8$的元素进行递归，所以算法的最坏情况时间复杂度满足：

$$W(n) \leq W\left(\left\lceil \frac{n}{7} \right\rceil \right) + W\left(\frac{5}{7}n + 8 \right) + O(n)$$

同样通过“guess and prove”的方式，我们不难证明$W(n) = O(n)$，具体证明留作习题。

上述证明表明了我们的猜测是准确的，将分组变得更细（从5个一组变成7个一组）并不会使算法的时间复杂度变坏。但是在大O的意义下，分7个一组也并不会对算法的时间复杂度带来本质改进，而只会让算法的具体实现变得更复杂。这是因为选择问题显然具有$\Omega(n)$的下界，而$O(n)$的SelWlin 算法已经是最优的。

以3和7为代表性的情况，我们对SelWlin算法中分组大小的影响进行了定量分析。上述分析表明，选择5个元素分为一小组是一个合适的选择。

第十章　对数时间查找 (L9)

授课提纲

查找概览

- 查找代价频谱
- 折半查找的概念与启示
- RB-Tree基本概念
- Hash表基本概念
- 并查集（Union-Find）基本概念

广义折半查找

- 经典折半查找再辨析：数组 vs. 链表
- 单峰的查找 $\Rightarrow$ 局部峰的查找
- value-index追及问题：least missing number；$A[i] = i$
- $\left\lfloor \sqrt{N} \right\rfloor$：$O(n^2)$，$O(n)$

第十一章　分治的平衡性 (T3)

授课提纲

- 平衡版的QuickSort
- maxima问题
  - 基于排序
  - 平衡分治
  - 一种有教学意义的错误分治
- 芯片查找问题
- 前k小的元素
  - $O(n \log n)$
  - $O(n + k \log n)$
  - $O(n + k^2)$（两种设计）
  - $O(n + k \log k)$
- 离median最近的k个数

第三部分

从图遍历到图优化

图遍历。

图优化：贪心、动态规划。

第十二章　图遍历 (L10)

授课提纲

图算法总览（§12.1）：

- 图算法问题：从遍历到优化；
- 图算法策略：从贪心到动态规划。

框架的基本概念：

- 共性内容的固化；
- 具体内容的细化：参数化、定制化。

精化（层层细化）观点下的图遍历：

- 不变式的概念（回顾）、精化的概念；
- 抽象的WFS框架（§12.2）；
- WFS ⇒ DFS、BFS、BestFS。

DFS框架基础：

- 抽象DFS框架；
- 嵌入application-specific处理的3组（共5个）时机。

DFS框架的增强：

- 点着色：W ⇒ G ⇒ B；
- 边着色：TE、BE、DE、CE；
- 时间的引入：StartTime、FinishTime、ActiveInterval。

12.1　图算法总览

首先我们列出图算法部分的关键知识点，如图12.1所示。然后我们从问题和策略两个正交的维度，来鸟瞰这些知识点。

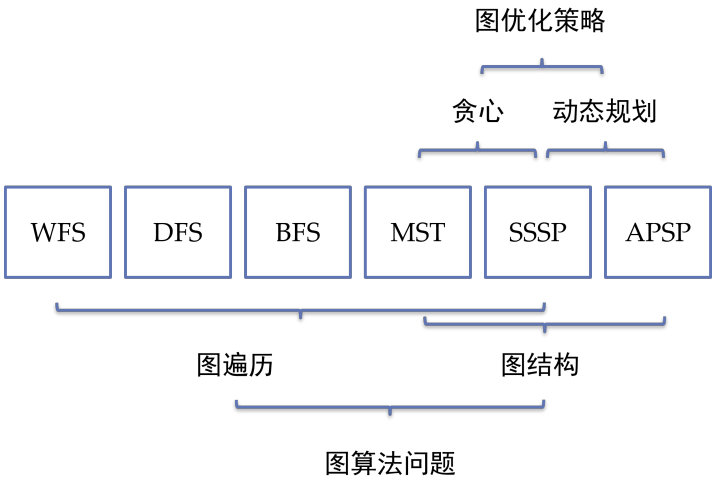


图 12.1: 图算法知识点总览

12.2　WFS遍历框架

“爱谁谁优先遍历”WhateverFirstSearch（WFS）：

- 基本做法：从一个点，顺着边，走到下一个点，直至走遍所有点和边。
- 设计要点：
  - 调度：抽象的bag结构（具体设计由后续的设计者来决定）；
  - 去重：点的二着色。

12.2.1　WFS的设计

WFS的设计，如算法1所示。 WFS的外层调用过程（wrapper）的设计，如算法2所示。

Algorithm 1 WFS(s)

```
1: put s into the bag B
2: while B is not empty do
3:     take v from B
4:     if v is unmarked then
5:         mark v
6:         for all edge vw do
7:             put w into B
```

12.2.2　从WFS逐步精化

以支撑遍历过程的“枢纽”性的数据结构为抓手，我们可以统一各类“遍历”算法，如图12.2所示。

Algorithm 2 WFSWrapper(G)

```
1: for all vertices v do unmark v                ▷ 遍历之前，初始化每个点
2: for all vertices v do
3:     if v is unmarked then WFS(v)
```

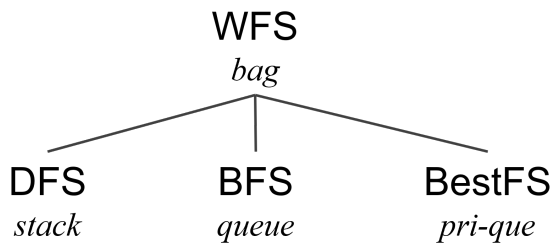


图 12.2: WFS框架的精化

第十三章　给定源点的最短路径

授课提纲

问题的定义：

- Path，Shortest Path (SP)。
- Single-source: SSSP。
- All-pair: APSP。

SSSP：

- 无权图：BFS。
- DAG：DFS（DP）。
- 非负权图：Dijkstra（负权图其实也行）。
- 负权图：Bellman-Ford（能检测负环）（§13.2）。

13.1　SSSP问题

13.1.1　关于负权边的subtleties

初看路径和最短路径的概念是直觉的、简单的。但是边是否有向、路径是否是简单路径、图中是否有负权边或者负权环，这些因素都会对这一表面简单的问题产生非常微妙的影响。具体而言：

- 当所有边权非负时，我们主要讲解有向图。但是有向图算法，稍加改动即可用于无向图。
- 当有负权边时，
  - 我们不考虑无向图。对于无向图，如果图中有负权边，问题会变得非常微妙。例如，一条负权无向边，如果简单替换为一对相向的有向边，则会引入一个负权环。含有负权边的无向最短路径的子路径不一定是最短路径。因此，从单个源顶点出发的所有无向最短路径的集合可能并不构成一棵树，即使最短路径是唯一的。我们不讨论含有负权边的无向图上的最短路径问题。

– 我们只考虑有向图。
  * 当有负权边时，如果有负权环，则SP问题不再是well-defined的。我们的核心问题变成了甄别出负权环。
  * 当有负权边时，如果没有负权环，我们可以考虑SP问题。

13.2　Bellman-Ford算法

13.2.1　Ford的generic框架

首先要做初始化。一方面，将源点s的$dist(s)$赋值为0，其它点u的$dist(u)$赋值为无穷。另一方面，源点s的$pred(s)$赋值为Null，并且根据定义，它一直为Null。其它点u的$pred(u)$初始化为Null。如果s到u存在最路径，则$pred(u)$将被赋值为最短路径上指向u的点（u前面的那个点）。初始化过程如过程InitSSSP(s)所示。

找到更短路径的关键，在于发现所谓紧（`tense`）的边。我们定义边$u \to v$为紧的，如果$dist(u)+w(u \to v) < dist(v)$。对于一条紧的边，我们可以Relax它，即更新最短路径值$dist()$，以及最短路径上的前一个点$pred()$，如过程Relax($u \to v$)所示。

有了紧的概念和Relax操作，我们可以得到朴素的蛮力SSSP算法，如算法**??**所示。这一框架是Ford 提出的。

```
1: procedure InitSSSP(s)
2:     dist(s) := 0
3:     pred(s) := NULL
4:     for all vertices v ≠ s do
5:         dist(v) := ∞
6:         pred(v) := NULL
```

```
1: procedure Relax(u → v)
2:     dist(v) := dist(u) + w(u → v)
3:     pred(v) := u
```

Algorithm 3 FordSSSP(s)

```
1: InitSSSP(s)
2: while there is at least one tense edge do
3:     Relax any tense edge
```

13.2.2　Ford框架视角下的Dijkstra算法

每个节点的状态都是从Fresh→ Fringe→Finished（本质就是BFS中的`white`→`gray`→`black`）。在Ford框架的视角下，这三种状态的含义是：

- Fresh：从未被Relax。
- Fringe：已经被Relax过，还可能再次（多次）Relax。
- Finished：已经完成Relax，再也不需要，也不可能被Relax。

13.3　从Ford到Bellman-Ford

13.3.1　Pre版本：细化边的遍历

Bellman-Ford算法就是对Ford框架的精化，它本质上也是一种蛮力算法，只不过对遍历和 Relax 每一条紧的边的过程，做了更细致的设计。

第一个抽象版本的Bellman-Ford算法（算法**??**）中，有两个要素。一个是对于紧的边的遍历，它明确设计成遍历图中的每一条边。至于具体如何遍历图中的边，是否要做各种优化，当前阶段的算法设计中尚未涉及。

另一个要素是对于寻找紧的边的过程如何终止，它被设计成一个简单但是有点“含糊”的while循环。

下面，我们首先就要细化、明确这个含糊的while循环。

Algorithm 4 BellmanFordPre(s)

```
1: InitSSSP(s)
2: while there is at least one tense edge do
3:     for every edge u → v do
4:         if u → v is tense then
5:             Relax(u → v)
```

13.3.2　Pre2版本：明确Relax的轮次

在Pre2版本中（算法**??**），含糊的、不知道多少轮的while循环，被替换成了明确的$n-1$次循环。循环的轮次被清晰定为$n-1$轮的道理在于，逐渐Relax的过程，就是小的路径，逐渐长成大的路径的过程，而最短路径至多有$n-1$跳，所以循环至多要$n-1$轮。上述论述只能说原则上是正确的，有一些细节需要补充，详细的证明过程可以参考[**?**]。

后续我们将引入“显式”（explicit）的递归刻画。基于这一详细的递归刻画，大家也可以严格地论证上述$n-1$轮迭代的正确性。

在本轮细化过程中，我们还显式地处理了负环的问题。如果没有负环，则SSSP问题被解决，则没有紧的边。做逆否命题的等价变换，我们有：如果还有紧的边，则有负环。负环问题的处理，我们在后续算法中不再专门写出来。有需要随时“插拔”即可。此外，处理负环的过程中，如何遍历所有的边，可以采用不同的实现方式。

Algorithm 5 BellmanFordPre2(s)

```
1: InitSSSP(s)
2: for i = 1 to n − 1 do
3:     for every edge u → v do
4:         if uv is tense then
5:             Relax(u → v)
6: for every edge u → v do
7:     if u → v is tense then
8:         return "negative cycle"
```

13.4　用DP细化Bellman-Ford算法的设计

13.4.1　从$dist(v)$到$dist_{\le i}(v)$

路径天然地具有“小路径逐步长大”的结构，所以围绕递归来设计最短路径算法，是一种非常直觉的选择。我们很直觉地可以想到下面的递归：

$$dist(v) = \begin{cases} 0 & \text{if } v = s \\ \min\limits_{u \to v}(dist(u) + w(u \to v)) & \text{otherwise} \end{cases} \tag{13.1}$$

但是这一递归却并不可行。因为如果u、v在一个环中的话，要求$dist(v)$，我们必须先知道$dist(u)$；而要求$dist(u)$，必须先知道$dist(v)$。这不是一个良定义的递归。

上述思路是可行的，只不过我们对于路径的刻画过于粗疏，使得相应的递归设计未能奏效。我们可以通过细化对于路径的刻画，进而得出更精细的、正确的递归设计。刻画路径的关键在于对于路径的大小（路径中边的条数）的界定。我们定义$dist_{\le i}(v)$为：在仅考虑不超过i条边的路径的前提下，s到v的最短路径的长度。由此，我们可以用细化后的$dist_{\le i}(v)$函数来刻画最短路径所满足的递归式：

$$dist_{\le i}(v) = \begin{cases} 0 & \text{if } i = 0 \text{ and } v = s \\ \infty & \text{if } i = 0 \text{ and } v \neq s \\ \min\left\{ \begin{array}{c} dist_{\le i-1}(v) \\ \min\limits_{u \to v}(dist_{\le i-1}(u) + w(u \to v)) \end{array} \right\} & \text{otherwise} \end{cases} \tag{13.2}$$

注意，上述思路利用了最短路径的一条非常重要的性质：更大的最短路径，一定是由更小的最短路径“拼接”而成的。也就是说，大问题的最优解，一定是小问题的最优解组装而成。这一性质并不是天然成立的。很多优化问题，我们可以用小问题的“次优解”，组装出大问题的最优解。

最短路径问题以及更一般的优化问题的这一性质，我们将在对动态规划策略做回顾总结和深度反思的时候，做专门讨论。

13.4.2　动态规划版的Bellman-Ford算法

根据$dist_{\leq i}(v)$所满足的递归式，我们可以得出用动态规划策略设计的BELLMAN-FORD算法，如算法**??**所示。

Algorithm 6 BELLMANFORDDP(s)

```
1: dist[0, s] := 0
2: for every vertex v ≠ s do
3:     dist[0, v] := ∞
4: for i = 1 to n − 1 do
5:     for every vertex v do
6:         dist[i, v] := dist[i − 1, v]
7:         for every edge u → v do
8:             if dist[i, v] > dist[i − 1, u] + w(u → v) then
9:                 dist[i, v] := dist[i − 1, u] + w(u → v)
```

13.4.3　两个细节的改进

最短路径权值$dist_{\leq i}(v)$的初始化，可以和边的RELAX分离开来，如算法**??**所示。

Algorithm 7 BELLMANFORDDP2(s)

```
1: dist[0, s] := 0
2: for every vertex v ≠ s do
3:     dist[0, v] := ∞
4: for i = 1 to n − 1 do
5:     for every vertex v do
6:         dist[i, v] := dist[i − 1, v]
7:     for every edge u → v do
8:         if dist[i, v] > dist[i − 1, u] + w(u → v) then
9:             dist[i, v] := dist[i − 1, u] + w(u → v)
```

提前访问到了更新后的$dist_{\leq i}(v)$值，至少不会有坏处，如算法**??**所示。

13.4.4　原地的Bellman-Ford算法

上述改进主要是让最后的“原地”（in-place）化改造变得更直观。

我们无需再维护$n-1$个矩阵，每个矩阵对应循环中的一个轮次。所有轮次直接在同一个矩阵上原地不断更新即可。

最终得到的Bellman-Ford算法，如算法**??**所示。

Algorithm 8 BellmanFordDP3(s)

```
1: dist[0, s] := 0
2: for every vertex v ≠ s do
3:     dist[0, v] := ∞
4: for i = 1 to n − 1 do
5:     for every vertex v do
6:         dist[i, v] := dist[i − 1, v]
7:     for every edge u → v do
8:         if dist[i, v] > dist[i, u] + w(u → v) then
9:             dist[i, v] := dist[i, u] + w(u → v)        ▷ 两处i − 1均改成了i
```

Algorithm 9 BellmanFord(s)

```
1: dist[s] := 0
2: for every vertex v ≠ s do
3:     dist[v] := ∞
4: for i = 1 to n − 1 do
5:     for every edge u → v do
6:         if dist[v] > dist[u] + w(u → v) then
7:             dist[v] := dist[u] + w(u → v)
```

第十四章　动态规划 (L19, L20)

授课提纲

-

14.1

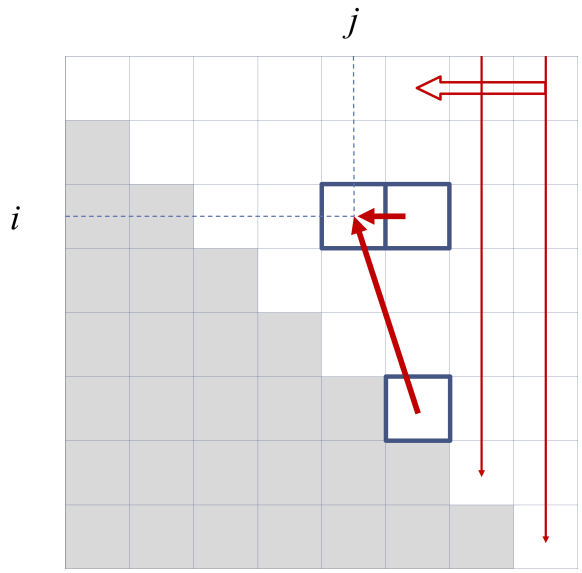


图 14.1: 子问题依赖关系图

14.1.1

14.1.2

Algorithm 10 LIS-In($A[1..n]$)

```
1: A[0] := −∞
2: for i = 0 to n do
3:     L_in[i, n + 1] := 0
4: for j = n downto 1 do
5:     for i = 0 to j − 1 do
6:         if A[i] ≥ A[j] then
7:             L_in[i, j] := L_in[i, j + 1]
8:         else
9:             L_in[i, j] := max(L_in[i, j + 1], 1 + L_in[j, j + 1])
10: return L_in[0, 1]
```

Algorithm 11 LIS-Out($A[1..n]$)

```
1: A[0] := −∞
2: for i = n downto 0 do
3:     L_out[i] := 1
4:     for j = i + 1 to n do
5:         if (A[j] > A[i]) and (1 + L_out[j] > L_out[i]) then
6:             L_out[i] := 1 + L_out[j]
7: return L_out[0] − 1
```

附录 A　阐述的结构

—— 如何让别人理解你的理解

按：我对“如何清晰陈述自己的研究内容，使之易于为他人所理解”这一问题的关注，始于从事研究、学习撰写论文之时。早期的相关训练，多来自师长的口授心传与个人的点滴体悟。随着研究资历渐深、讲授经验渐丰，关于“如何把一个事物阐述清楚、如何让听者顺畅地接受你的阐述”，我逐渐积形成了自己的理解与阐述方式。

理解渐趋成熟时，我同时意识到：此类观点，乃至相应的方法与技术，在既有的哲学、心理学与教育学中，应当早已存在前人较为成熟、系统的论述。然而在大模型出现之前，我对这类知识的获取，大抵限于网络上的零星涉猎与不成体系的学习；也曾有意寻访相关著作，但资料的检索、汇总与系统化研习——尤其是考虑时间成本——实为不易。

如今借助大模型，我可以先陈述自己的观点，请其调研相关理论，再择取合适者加以整理，支撑和充实我既有的理解。本文的写作正是如此：我选择了这个问题，并且给出了我自己体悟到的基本框架，大模型为其配匹既有的理论体系并形成更为详尽的论述。

需要说明的是，本文不少地方本来可以、也应当引证更为严肃而丰富的参考文献；但由于我并未在严格的学术意义上研读相关文献，故仅择取少数密切相关的经典著作，列为“延伸阅读”附于篇末，以供参考。我在需要时，亦应做更深入、更全面的阅读。

A.1　引言——从一个普遍的困境出发

你掌握了一门知识，自认已经理解得很透彻。但当别人问你“能不能给我讲讲”时，你往往会陷入两种失败：

- 倾倒式失败：把你知道的全部细节一股脑倒出来，对方眼神涣散，最后说“你讲得太细了，我还是没明白”；
- 跳跃式失败：默认对方和自己有同样的背景，不断地通过“指针”指向你认为对方必定知道的既有引理，三言两语跳到结论，对方一头雾水，你则困惑“这不是很显然吗”。

这两种失败的共同点在于：你把“阐述”当成了“信息的搬运”——好像知识是一个包裹，递过去就算完成。但理解的真相是：听者从来不是在“接收”信息，而是在“建构”理解。阐述者真正要做的，不是倾倒，而是为对方重建理解的结构。

本文用三个相互衔接的理论回答一个问题：为什么有效的阐述应该是“先框架、后细节”的层次结构，以及怎样做到这一点。

- 理解的前结构（海德格尔）——从哲学上回答：为什么听者不可能“从零接收”；
- 认知负荷理论（斯威勒）——从心理学上回答：为什么不能一次给太多细节；
- 结构主义教学理论（布鲁纳）——从教育学上回答：具体应该怎样组织阐述。

三个理论分别对应阐述活动的三个层面：理解的逻辑、记忆的限制、操作的工程。

A.2　理解的前结构——听者从来不是一张白纸

关键词：前有（*Vorhabe*）、前见（*Vorsicht*）、前把握（*Vorgriff*）、视域融合

A.2.1　核心命题

海德格尔在《存在与时间》中提出：任何理解都始于“前结构”。人在理解任何事物之前，已经携带了三样东西，如下表A.1所示：

表 A.1: 理解前结构的三个组成部分

概念	含义	一个类比
前有（Vorhabe）	你所置身的背景：文化、语言、历史、生活经验累积而成的“世界”	出发时身上已有的行李
前见（Vorsicht）	你所采取的视角：带着特定观点、目的或问题意识去切入	你朝哪个方向看
前把握（Vorgriff）	你预先抱有的概念框架：先用已有逻辑去“套”新经验	你手里的地图

由此得出一个对阐述至关重要的推论：“从零开始”的理解是不可能的。如果一个人真的剥离了全部前结构，他就同时失去了语言、尺度和坐标系——面对完全陌生的事物，他不仅无从下手，甚至无法意识到“这是一个需要被理解的东西”。

A.2.2　对阐述的启示

这条哲学原理直接规定了阐述者的第一项义务：在传递任何新信息之前，先为对方搭好前结构。 具体说：

1. 前结构不是妨碍，而是通道。 讲解者先给出的那个“抽象框架”，就是为听者准备的“前有”和“前把握”。没有它，细节对听者只是无意义的信息碎片——这正是“听的时候好像懂了，回头全忘了”的机制：碎片从未进入对方的理解结构。
2. 要诊断对方已有的前结构。 阐述前先弄清：对方已经知道什么？用什么视角看世界？手里是哪张地图？同一段话，对数学系学生和文学系学生的有效讲法完全不同，因为两者的“前见”不同。

3. 框架先行不是降低标准，而是理解的必经通道。 伽达默尔继承海德格尔，把这一思想发展为“视域融合”：理解发生在讲解者的视域与听者的视域相互融合之处。框架，就是两块视域的接触面。

小结：阐述的起点不是“我说了什么”，而是“对方站在哪里”。

A.3　认知负荷理论——工作记忆是瓶颈

关键词：工作记忆、长时记忆、图式、三类认知负荷、组块化

A.3.1　核心命题

认知负荷理论（Cognitive Load Theory）是三个理论中实证基础最强的一个。它的出发点是一个残酷的事实：

- 工作记忆容量极小——通常只能同时处理 2～4 个信息组块，持续时间仅几秒钟，是学习的瓶颈；
- 长时记忆容量几乎无限，它以“图式”（schema，即组织化的知识结构）的形式储存信息，并借助图式大幅降低实际加工成本。

学习的本质，是让新信息经由工作记忆、被整合进长时记忆中的图式。工作记忆过不去，一切免谈。

A.3.2　三类认知负荷

学习者在加工材料时承受的负荷由三部分构成：

1. 内在认知负荷（*intrinsic load*）：由材料本身的复杂程度和听者的已有水平决定。材料越难、听者基础越差，内在负荷越高——这部分是“客观成本”，只能管理，不能消除；
2. 外在认知负荷（*extraneous load*）：由糟糕的呈现方式造成——杂乱的排版、冗余的表述、与目标无关的信息。这部分负荷对学习毫无益处，必须尽力消除；
3. 相关认知负荷（*germane load*）：用于建构和自动化图式的有效认知投入。优秀的阐述应当把省下来的资源引导到这一部分。

一句话概括：把外在负荷压到最低，把内在负荷拆散分批，把相关负荷喂到最足。

A.3.3　对阐述的启示

1. 先给骨架，再分层加载细节。 先用少量元素建立图式，占据最少的工作记忆；此后每一层细节都“挂”在已建好的图式上，被当作一个组块处理。若一上来就倾倒全部细节，会立即超出工作记忆容量——听者既记不住细节，也抓不住结构。
2. 组块化（*chunking*）。 把复杂知识拆成小模块分步呈现。检验标准很朴素：一次只讲“几件事”，这个“几”应当不超过 3～4 个。

3. 警惕外在负荷。 啰嗦的铺垫、跑题的例子、炫技的术语、没有信息量的幻灯片动画——它们都在占用听者宝贵的工作记忆。省下来的容量，才能用于真正理解你讲的内容。

4. 利用示例效应。 对初学者，给出完整示范（worked example）比直接扔出难题让他自己摸索有效得多。讲概念之前，先给一个走通全程的具体例子。

小结：听者的工作记忆是一条狭窄的通道。阐述者的克制——少说一点、分层说、说干净——不是风格问题，而是认知工程学。

A.4　结构主义教学理论——把阐述做成工程

关键词：学科基本结构、螺旋课程、三种表征、四条教学原则

A.4.1　核心命题

如果说前两讲回答了“为什么”，布鲁纳的结构主义教学理论回答的是“怎么做”。布鲁纳（Jerome Bruner）主张：教学的中心不是传授事实，而是使学生掌握学科的基本结构——一门学科的基本概念、基本原理、基本态度和方法。

为什么结构优先？因为掌握基本结构有三重回报：

- 更容易理解：细节可以推导，结构不能；
- 更容易记忆：结构化的知识以组块形式储存，占用更少认知资源（这正是认知负荷理论的心理学基础）；
- 更容易迁移：面对新问题时，结构是唯一能带得走的东西。

布鲁纳有一句名言：“任何学科的基本原理，都可以用某种形式教给任何年龄的任何人”——前提是按正确的层次组织。

A.4.2　两个关键机制

（*1*）螺旋课程（*spiral curriculum*）

同一个主题，反复出现，每次在更高的抽象层次上深化。这给出了“逐层展开”的标准节奏：第一圈给轮廓（不求全对，但求有图），第二圈修正和加深，第三圈加入例外与边界。物理学教学就是范本：质点 $\rightarrow$ 刚体 $\rightarrow$ 弹性体 $\rightarrow$ 连续介质，每一层都是前一层的结构化升级。

（*2*）三种表征系统

人的认知有三种表征方式：动作表征（靠做）、映像表征（靠图）、符号表征（靠语言和公式）。一个完整的阐述应当让同一个结构在三种表征中各走一遍——能画图就不用纯文字，能演示就不只描述。这也是降低外在认知负荷的手段。

A.4.3　四条教学原则 $\rightarrow$ 阐述四原则

布鲁纳的四条教学原则可以直接转写为阐述的操作清单，如表A.2所示。

小结：布鲁纳把“先框架后细节”从一种讲法升格为一套可执行的工程规范——结构优先、螺旋深入、多表征呈现、逐层反馈。

表 A.2: 布鲁纳的教学原则

教学原则	阐述中的对应动作
动机原则	开场先建立“为什么值得听”——抛出真实问题或认知冲突，而不是先定义术语
结构原则	先呈现学科/主题的基本结构，而不是罗列事实；时刻回答“这部分在整个图景中处于什么位置”
程序原则	安排材料的先后顺序：由浅入深、由整体到局部、由已知到未知，节奏上先轮廓后细节
强化原则	每讲完一层，立刻用反馈检验：让对方复述、提问或做题，确认这一层已挂牢，再进入下一层

A.5　总结——三个理论构成一个闭环

把三条线索串起来，就是本文的全部结论：

1. 因为理解必须有前结构（哲学），所以框架先行；
2. 因为工作记忆容量有限（心理学），所以细节必须分批挂靠；
3. 因为结构可以被工程化地组织（教育学），所以这套做法可以学习、可以复制。

三条线索的总结，以及对于阐述者的操作性建议，如表A.3所示。

“先粗后细、先抽象后具体”因此不是讲解者的个人风格，而是对人类认知结构的顺应——它同时满足了理解的逻辑、记忆的限制和交流的经济性。好的阐述，是让听者在每一层都站在坚实的地基上，再邀请他向上一步。

表 A.3: 总结与建议

	回答的问题	核心命题	给阐述者的提示
理解的前结构	理解的逻辑	听者不可能从零接收，理解必须借助先行框架	先为对方搭好“前有—前见—前把握”，再谈新内容
认知负荷理论	记忆的限制	工作记忆只有 2–4 个组块，细节必须挂靠图式	分层投放、组块化、清除外在负荷、善用示例
结构主义教学理论	操作的工程	先呈现基本结构，螺旋式逐步深化	结构优先、螺旋展开、多表征、逐层反馈

进一步阅读

1. 海德格尔. 存在与时间（修订译本）[M]. 陈嘉映, 王庆节, 译. 北京: 生活·读书·新知三联书店, 2006.
2. 伽达默尔. 真理与方法：哲学诠释学的基本特征（诠释学 I ）[M]. 洪汉鼎, 译. 上海: 上海译文出版社, 1999.

3. John Sweller, Paul Ayres, Slava Kalyuga, Cognitive Load Theory[M]. Springer, 2011.

4. 布鲁纳. 教育过程[M]. 邵瑞珍, 译. 北京: 文化教育出版社, 1982.

参考文献

[1] list_sort.c in the Linux kernel. `https://github.com/torvalds/linux/blob/master/lib/list_sort.c`. Last accessed: Sep. 2026.

[2] Richard W. Hamming. *The Art of Doing Science and Engineering: Learning to Learn*. Taylor & Francis, 2005.

[3] Jon Kleinberg and Eva Tardos. *Algorithm Design*. Prentice Hall, 2006.

[4] Wikipedia. Karatsuba algorithm. `https://en.wikipedia.org/wiki/Karatsuba_algorithm`. Last accessed: Sep. 2026.

[5] Wikipedia. Merge sort. `https://en.wikipedia.org/wiki/Merge_sort`. Last accessed: Sep. 2026.